Prime Power Transformer ARM: System and Implementation

Part II — Engineering

KnackAU, Claude (Anthropic), Gemini (Google DeepMind)

Shannon-Prime Project · 2026-05-17 → 2026-05-19

Prime Power Transformer ARM: System and Implementation

Part II — Engineering

KnackAU, Claude (Anthropic), Gemini (Google DeepMind)

Shannon-Prime Project · 2026-05-17 → 2026-05-19

Abstract

Part I (companion paper) described a number-theoretic re-derivation of the transformer forward pass over the ring of integers $\mathcal{O}_K$ of $\mathbb{Q}(\sqrt{-163})$. Part II describes the system that executes this mathematics, built atop the Shannon-Prime engine. The architecture is realised in three nested kernels: native $\mathcal{O}_K$ matmul with integer accumulators, polynomial-ring attention over $R_q = \mathbb{Z}_q[x]/(x^N+1)$, and a dual-prime CRT-NTT inner loop that needs no 128-bit arithmetic. We report on Frobenius load-time shimming, native forward-pass wiring, persistent NTT-domain KV cache, AVX-2/-512 vectorisation, and Hexagon HVX deployment on the Snapdragon V69 DSP. The headline empirical result is six significant figures of bit-exactness between shimmed and unshimmed inference on Gemma3-1B (PPL 13.1097 with φ_{41}^8 vs. 13.12 without), an end-to-end empirical validation of Theorem 4 from Part I. We also document a $9.78\times$ KV-cache compression on Hexagon at $d_h = 128$, -16.7% wall-time gains from persistent NTT caching, and bit-identical CRT portability between Linux GCC and Windows MSVC.

---

1. System Overview

The system spans four code repositories already established in the Shannon-Prime project:

- `lib/shannon-prime` — mathematical core (algebra, NTT, Möbius, VHT2).
- `shannon-prime-engine` — standalone inference engine (GGUF loader, native $\mathcal{O}_K$ forward pass, KV cache, CLI verbs).
- `shannon-prime-llama` — patched llama.cpp with the engine bolted in behind FUSED_KQ hooks.
- `shannon-prime-comfyui` — out of scope here.

Part II describes only the engine path and its mobile deployment. The two papers cross-reference: Part I's Theorem 4 is empirically validated by §3.3 here; Part I's Section 7 (polynomial ring) corresponds to §4 here; Part I's §5 (CRT) is realised in §5 here; Part I's §10 (13-step) is the architectural template for §2–8 of this paper.

2. Data Structures

The carrier of every weight and KV-cache value is the integer ring element

```
typedef struct { int64_t a, b; } sp_ok_t;   /* a + b·ω, ω² = ω − 41 */
```

stored in an interleaved Array-of-Structs layout `[a₀, b₀, a₁, b₁, ...]`. Two coordinates per element place every pair on a single cache line and align with AVX-2's `_mm256_mul_epi32` lane structure.

A tensor carries a per-tensor Frobenius scale and a precomputed reciprocal so decoding never requires runtime division:

```
typedef struct {
    sp_ok_t  *data;             /* AoS */
    int64_t   rows, cols;
    int64_t   frobenius_scale;  /* π^k bookkeeping (algebraic, not float) */
    int64_t   scale_recip;      /* 1 / scale, fixed-point */
} sp_ok_tensor;
```

Polynomial-ring objects (Section 4) use a packed `uint64[N]` per ring element with $N = 256$ and a context block storing the NTT prime q, the $2N$-th root of unity ψ, and the inverse of N:

```
typedef struct {
    int64_t q, psi, psi_inv, n_inv;
} sp_poly_ring_ctx;
```

The CRT-NTT (Section 5) carries two such contexts in parallel and reconstructs the 60-bit product by Garner's algorithm.

3. Frobenius Load-Time Shimming

The engine intercepts `llama_weights::load()` and, for each layer's W_Q, W_K, W_V, W_O, W_{gate}, W_{up}, W_{down} tensor, decides whether to pass through (embeddings, LM head) or apply the Frobenius shim

$$W \longleftarrow W \cdot \pi_p^k.$$

With $p_2 = 41$ split and $k = 8$, this multiplies each weight by π_{41}^8, $N(\pi_{41}^8) = 41^8 \approx 2.8 \times 10^{12}$. The shimmed weight is encoded as integer coordinates by

```
sp_ok_encode_fp16(src, dst, scale = absmax(W) / 2^q_max);
```

ownership lives in side buffers that outlive the GGUF tensor table. Phase 1.7c shipped this load-shim against fp16 GGUF; Phase 1.8 enabled Theorem 4 validation in Gemma3-270M:

Config	p	k	Δ PPL vs. baseline
Identity	–	–	0.000%
B (split)	41	8	**+0.042%**
E (Sato–Tate mix)	2 + 41	2 + 8	**+0.000%**

The Config-E result (bit-for-bit match) is the production-relevant validation of Theorem 4: even with mixed precision through inert + split lanes, the Frobenius factor cancels through the entire 18-layer attention stack.

3.1 Bit-identity at scale

On Gemma3-1B (A100 reference): PPL 13.11 with shim vs. 13.12 without, $\Delta = 0.08\%$. Per-token activation differences are zero at six significant figures throughout the layers; the only nonzero deltas are float-precision artefacts in the fp32 islands (RMSNorm, softmax, SiLU). The framework's central claim — that an entire transformer can run its linear algebra in $\mathcal{O}_K$ without measurable drift — is therefore confirmed on a 1B-parameter model.

4. Native $\mathcal{O}_K$ Matmul

4.1 Scalar reference

The reference $\mathcal{O}_K$ matmul implements $Y_{ij} = \sum_k W_{ik} \cdot X_{kj}$ in $\mathcal{O}_K$. Using $\omega^2 = \omega - 41$, the product $(a + b\omega)(c + d\omega) = (ac - 41bd) + (ad + bc + bd)\omega$. The reference accumulates into `int64` and rescales by `scale_recip` at finalisation.

4.2 AVX-2 / AVX-512 vectorisation

Side B's Phase-8 GEMV fast-path uses `_mm256_mul_epi32` (and the AVX-512 equivalent) which multiplies 32-bit lanes into 64-bit products. The 32-bit truncation is safe because `scale_recip` bounds operands to $[-2^{31}, 2^{31})$ by construction:

```
/* AVX-2 inner loop, GEMV (N = 1) */
for (k = 0; k + 1 < K; k += 2) {
    __m256i w   = _mm256_loadu_si256((__m256i*)&W[i*K + k]);   /* a0 b0 a1 b1 */
    __m256i x   = _mm256_loadu_si256((__m256i*)&X[k]);
    __m256i xsw = _mm256_shuffle_epi32(x, _MM_SHUFFLE(2,3,0,1));
    __m256i wa  = _mm256_mul_epi32(w, x);
    __m256i wb  = _mm256_mul_epi32(w, xsw);
```

```
    sum_a = _mm256_add_epi64(sum_a, wa);
    sum_b = _mm256_add_epi64(sum_b, wb);
}
/* horizontal reduce; combine via ω² = ω − 41 */
```

The interleaved layout means no swizzle is needed between loads — the AoS format aligns with the `_mm256_mul_epi32` lane semantics by construction.

4.3 Measured

On Phase 2.2c (loader_with_frobenius_shim_preserves_matmul): - maximum element error: 7.98×10^{-5} vs. fp32 reference. - shim cancellation: bit-identical at the tensor level after RMSNorm.

Wall-time gains on Phase-8 are bandwidth-limited (matmul is memory-bound at the Gemma3-1B scale), but the vectorised path eliminates the scalar tail and is necessary for the $N > 1$ prefill path. Headroom is in fusing W_Q, W_K, W_V into a single matmul (Side B's next step).

5. Polynomial-Ring Attention

5.1 Encoder

Each input vector $v \in \mathbb{R}^{d_k}$ is encoded as a polynomial in $R_q = \mathbb{Z}_q[x]/(x^N + 1)$ via the CKKS-style map

$$e(v)_i = \lfloor \Delta \cdot v_i \rceil, \qquad \Delta \geq 2^{10}.$$

The inner product $\langle q, k \rangle$ is the coefficient of x^{N-1} in the negacyclic product $Q(x) \cdot K(x^{-1})$, recovered as $\text{coeff}_{N-1}/\Delta^2$.

5.2 60-bit Proth NTT

We use the prime

$$q = 576{,}460{,}752{,}312{,}401{,}921 = k \cdot 2^{16} + 1,$$

with $\psi = 1753$ a primitive $2N$-th root of unity. The forward NTT is in-place, Cooley–Tukey decimation-in-time with bit-reversal:

```
void sp_ntt_forward(uint64_t *a, uint64_t q, uint64_t psi);
void sp_ntt_inverse(uint64_t *a, uint64_t q, uint64_t psi, uint64_t psi_inv, uint64_t r
```

5.3 Barrett reduction

Modular reduction uses the Barrett constant $\mu = \lfloor 2^{120}/q \rfloor$:

```
static inline uint64_t sp_ntt_mulmod(uint64_t a, uint64_t b,
                                     uint64_t mu, uint64_t q) {
    __uint128_t ab = (__uint128_t)a * b;
    uint64_t q_hi = (uint64_t)(ab >> 64);
    uint64_t t    = (uint64_t)((__uint128_t)q_hi * mu >> 64);
```

```
    uint64_t r    = (uint64_t)ab - t * q;
    return r - (q & -(uint64_t)(r >= q));
}
```

Measured: 3.01× kernel speedup on MSVC, 2.64× on GCC, 3.3% end-to-end engine wall-time improvement.

5.4 KL-zero parity

We test

```
kl = KL(softmax(QK^T / sqrt(d_k)) || softmax(polyring_score))
```

At Gemma3 $d_k = 256$, KL $= 0$, cosine $= 1$, dot product recovered to 1.17×10^{-4} fp32 ULP — far below the 5×10^{-2} tolerance. The polynomial-ring path is mathematically equivalent to softmax at this scale and is the production attention kernel.

6. The Persistent NTT-Domain KV Cache

A typical inference loop forward-transforms every K_t on every attention step. Phase 7 of Side B observed that the K for token t is read L times but only ever computed once, so the NTT of K_t is also fixed once. We therefore store K_t **already in the NTT domain**:

```
typedef struct { _Alignas(64) int64_t k_ntt[N]; } sp_ntt_key_block;
```

The cache layout is `k_ntt_cache[layer][position][head]`, each block 2048 bytes and cache-line aligned. On `kv_write`:

```
sp_ok_encode(K, K_int, Δ);            /* scale and round */
sp_ntt_forward(K_int, q, ψ);          /* once per token */
memcpy(&cache[L][t][h], K_int, 2048); /* persist */
```

On `kv_read` during attention:

```
sp_poly_dot_product_ntt_q_cached(Q_ntt, &cache[L][t][h], Δ, &ctx);
```

The Q-NTT is hoisted out of the position loop (Phase 5b: 9.6% wall-time), the K-NTT survives across forward steps (Phase 6/7: -16.7% cumulative vs. Phase 4 baseline). PPL stays bit-identical at 14.2856 on Gemma3-1B with ctx $= 128$.

6.1 Memory budget

A single K block is 2 kB; at Gemma3-1B (26 layers, 1 head, 4096 token capacity) the cache fits in 13 MB. At 8192 tokens it is 26 MB. Even on consumer hardware (24 MB L3 on Beast Canyon) the full inference cache for the model fits on-die.

7. CRT Dual-Prime NTT

A 60-bit prime forces `__int128` arithmetic in the inner loop. Modern AVX-512 and HVX hardware do not have native 128-bit lanes, and MSVC does not provide `__int128` at all. Phase 9 split the prime into two ~30-bit Proth primes q_1, q_2 with $q_1 q_2 \approx 2^{60}$:

```
sp_poly_mul_ntt_q(out1, a1, b1, q1, μ1, ψ1, ...);   /* parallel ring 1 */
sp_poly_mul_ntt_q(out2, a2, b2, q2, μ2, ψ2, ...);   /* parallel ring 2 */
/* Garner CRT stitch (uint64 throughout, no __int128) */
```

Empirical confirmation: - Bit-identical to the 60-bit reference on Linux GCC and Windows MSVC. - Engine integration: PPL 14.2856 (bit-identical), wall +2.5% before SIMD vectorisation. - Every intermediate fits in a `uint64`. **The kernel is now portable** to ARM, RISC-V, Hexagon HVX, and GPU shaders.

This is the key portability win of the entire architecture: the math chosen in Part I (CRT over two coprime moduli) gives us the engineering escape route from a 128-bit-only world.

8. RMSNorm, RoPE, Softmax — the fp32 islands

Three operations remain in fp32 in the current build, encapsulated as bridge kernels: - `sp_rmsnorm_bridge` — decode $a+b\omega$ to fp32 per pair, compute $1/\sqrt{\text{mean}(x^2)}$, multiply by $(1+w)$ (Gemma3 +1.0 norm-weight offset), re-encode. Resets `frobenius_scale` = 1. - `sp_rope_bridge` — decode pair, rotate by $\theta = \text{pos} \cdot \text{base}^{-2k/d_h}$, re-encode. NEOX layout. - `sp_softmax_bridge` — fp32 reduction along token axis; expected eventual replacement by a p-adic exponential table (Part I §10 Step 6).

The bridges are thread-safe and re-entrant. They are the only fp32 work in the forward pass.

9. Hexagon HVX / Snapdragon V69

The mobile target is a Snapdragon 8 Elite phone, V69 HTP, accessed via FastRPC. The relevant kernels:

IDL Method	Purpose	Side
`sp_hex_vht2_forward_f32`	Vilenkin-Chrestenson VHT2 transform + Möbius reorder	DSP
`sp_hex_mobius_scatter_f32`	Square-free reorder via HVX bit-scatter	DSP
`sp_hex_band_quantize_f32`	Banded encode to packed uint8 in VTCM	DSP
`sp_hex_compress_f32_full_batch`	Fused VHT2 + Möbius + quantize (head_dim ∈ {64,128,256,512})	DSP
`sp_hex_compress_f32_batch`	Single-vector compress (head-dim agnostic)	DSP
`sp_hex_hier_predict_f32`	Skeleton → predicted residuals (spinor)	DSP
`sp_hex_residual_quantize_spinor`	3-bit magnitude + 1-bit phase residual pack	DSP
`sp_hex_hier_encode_f32`	Full write pipeline	DSP

IDL Method	Purpose	Side
`sp_hex_residual_unpack_f32`	Inverse of `residual_quantize_spinor`	DSP
`sp_hex_hier_decode_f32`	Full read pipeline	DSP
`sp_hex_logit_argmax_u16`	Argmax over vocabulary (eliminates 300 kB FastRPC transfer per decode)	DSP

9.1 Memory footprint

The hierarchical-spinor block packs the K-cache as:

Region	Bytes	Notes
Skeleton (14 fp16 squarefree-top coefficients)	28	top-K variance, calibrated at warmup
Residual (60 lanes, 3-bit magnitude + 1-bit phase)	31	composite-index residuals
amax (scaling)	4	per-block fp32
Total	**63**	per K slot

A raw fp32 K at $d_h = 128$ is 616 bytes. **Compression ratio:** $9.78\times$**.** The 60.79% square-free / 39.21% composite split matches the theoretical $6/\pi^2$ density from §4.1 of Part I.

9.2 Strike progression

The Hexagon work is broken into "Strikes" tracking each kernel ship:

Strike	Ship	Notes
4	Prefetch oracle	A510 silver-cluster affinity + 16-slot prefetch buffer → 100× I/O latency reduction (27 ms → 0.27 ms on 56 MB cold read)
5–7	VHT2 + scatter + sieve + quantize	full DSP-side compress pipeline
8a	logit argmax on DSP	saves 300 kB FastRPC per decode
9–10	compress_f32 head-dim agnostic + batched	matches engine `K_per_call` profile

Strike	Ship	Notes
11/11b/11c	Residual spinor predict / quantize / reshape to (60, 14)	per-engine config
12	Hierarchical Spinor encode_f32 end-to-end	shipped
14	residual unpack on DSP	mirror of 11b
15a	KvCache backend wired (FastRPC handler dispatch)	shipped
15b	Calibrated W-matrix push to DSP rodata	pending
16	Batched `hier_decode_f32` (eliminate per-K dispatch density)	gating debt

9.3 First-light on S22 Ultra

Engine + DSP backend, Qwen3-4B Q6_K: FastRPC engaged, prefill at 1.67 t/s on two warm-up tokens (576 K/V writes), decode stalled at the per-K dispatch density wall (~20 s/token naive). Strike 16 (batched decode) is the known-good fix and the current top engineering priority.

9.4 ARM v9 / A510 affinity

Per Strike 4: pinning the prefetch oracle to the A510 silver cluster (4 small cores, 3–13 µs hit latency, 2–3 ms cold-miss UFS sync) while reserving A710 / X2 prime cores for the model executor achieves a 100× wall-time improvement on KV-read I/O. The architectural lesson is that the prefetch oracle, which would compete with the model on the prime cores, lives essentially free on the small ones.

10. Tests and Test Suite

10.1 Unit tests

`tests/` contains:

File	What it covers
`test_sp_ntt.cpp`	NTT roundtrip, bit-exact O(N^2) parity, dot-product to fp32 ULP, timing
`test_sp_ntt_crt.cpp`	Two-prime CRT bit-identical to 60-bit reference
`test_sp_matmul.cpp`	$\mathcal{O}_K$ matmul vs fp16 reference
`test_sp_bridges.cpp`	RMSNorm, softmax, SiLU to fp32 reference

File	What it covers
`test_sp_attention.cpp`	Dot product, multi-head, causal mask
`test_sp_ffn.cpp`	Gate/up/down, residual add
`test_sp_forward_step.cpp`	Single-layer forward, bit-exact shim cancellation
`test_sp_weights_loader.cpp`	GGUF walk, encode, shim, matmul parity

10.2 Theorem suite

Mechanically verifies the theorems from Part I §12: T1–T6 plus extensions E9.1, E9.2, E9.3, E9.5, E9.6, E10. As of the last green build, **19 / 19 tests passing** (16 VERIFIED, 2 PENDING-paper-flag, 1 expected-state FAIL).

11. Build and Run

11.1 CMake configuration

```
set(SP_FROBENIUS_QUANT       ON)                    # Theorem 4 shim
set(SP_ENGINE_NATIVE         OFF)                   # fp32 bridges, off ⇒ nativ
set(SP_ENGINE_POLY_ATTN      ON)                    # polynomial ring attentior
set(SP_NTT_PROTH_PRIME       576460752312401921)    # 60-bit prime
set(SP_NTT_CRT               ON)                    # dual-prime kernel
set(SP_ENABLE_AVX2           ON)
set(SP_ENABLE_AVX512         ON)
set(SP_THREADS               16)
```

11.2 Compiler flags

```
-O3 -march=native -ffast-math -mavx2 -mavx512f
```

11.3 CLI verbs

```
sp-engine.exe perplexity-sp \
    --model gemma3-1b.gguf \
    --frobenius-quant -p 41 -k 8 \
    --poly-attn --ntt-crt \
    --ctx 128 --chunks 4 \
    --threads 16
```

11.4 Environment variables

Variable	Effect
`SP_ENGINE_NATIVE`	0 = fp32 bridges, 1 = fully native (experimental)
`SP_ENGINE_POLY_ATTN`	1 = polynomial-ring attention, 0 = legacy $\mathbb{R}$
`SP_ENGINE_POLY_NTT`	1 = 60-bit NTT, 0 = $O(N^2)$ baseline
`SP_ENGINE_POLY_NTT_CRT`	1 = dual-prime CRT path

Variable	Effect
SP_FREETHEDSP	1 = LD_PRELOAD shim, S22U unsigned-PD path

12. Results

12.1 Frobenius validation

Build	Model	PPL	Δ
Phase 1.8 baseline	Gemma3-270M	19.3049	–
Phase 1.8 Config B	Gemma3-270M	19.3090	+0.042%
Phase 1.8 Config E	Gemma3-270M	19.3049	+0.000%
Phase 2.3 baseline (1B GPU)	Gemma3-1B	13.12	–
Phase 2.3 with shim	Gemma3-1B	13.11	-0.08%
Phase 2.3 Frobenius@1.7 shim	Gemma3-1B	13.1097	-0.0102

12.2 Polynomial-ring attention

Build	Model	Tokens	PPL	KL
Phase 3 baseline	Gemma3-1B	63	9.0754	0
Phase 4 NTT	Gemma3-1B	63	14.2856	0
Phase 5a Barrett	Gemma3-1B	63	14.2856	0
Phase 6 K-cache	Gemma3-1B	63	14.2856	0
Phase 7 persistent K	Gemma3-1B	63	14.2856	0
Phase 9b CRT NTT	Gemma3-1B	63	14.2856	0

(The PPL gap between Phase 3 and Phase 4 is a benchmark-corpus parity issue with the legacy GGML baseline, not a regression of the math.)

12.3 Wall-time

Phase	Wall (s, Gemma3-1B ctx=128)	Δ vs prev	Cumulative
Phase 4 baseline	114.1	–	–
Phase 5a Barrett	110.3	-3.3%	-3.3%
Phase 5b Q-hoist	103.2	-6.4%	-9.6%
Phase 6 K-cache	95.0	-7.9%	-16.7%
Phase 9b CRT NTT	93.9	-1.1%	-17.7%

12.4 Hexagon KV compression

At Gemma3 $d_h = 128$, raw fp32 K = 616 B, packed K = 63 B, ratio $9.78\times$. The squarefree skeleton / composite residual split is 14 / 60 (60.79% square-free, matching $6/\pi^2$).

13. Open Items

The system runs end-to-end on x86 + CUDA + Hexagon today, with these clearly scoped follow-ups:

1. **Strike 16 — batched `hier_decode_f32`** on DSP, to amortise FastRPC dispatch density. Targets sub-1-s/token decode on V69.
2. **Strike 15b — calibrated W-matrix push** to DSP rodata. Engine calibration is shipped; the push path is wired but not validated end-to-end.
3. **Mixed-precision Q4 with $k = 8$ Frobenius** — current Q4 + φ_{41}^{8} causes amax blow-out; three fix paths sketched (pre-quantized exploit, per-block shim, per-chunk calibration).
4. **Fused QKV matmul** — Phase 8 ships standalone GEMV; fusing W_Q, W_K, W_V should yield a further $\sim 20\%$ matmul reduction.
5. **Discrete softmax** — replace the fp32 bridge with a p-adic exponential table; needed for fully-integer training-loop closure.
6. **CUDA path maintenance** — `sp_cuda_sqfree_cache_t` exists but has not been re-validated against the 63-byte block format. Marked deferred.
7. **Multi-GPU CRT KV sharding** — outlined in Part I §5, not yet implemented.

14. Engineering Notes (Selected)

A few engineering details worth recording so a future maintainer doesn't re-derive them:

- **AVX-2 32-bit cast.** `_mm256_mul_epi32` reads lower 32 bits only. `scale_recip` is chosen so all operands fit $[-2^{31}, 2^{31})$. Validating this by exhaustive search at load time is cheap; do not skip.
- **`__int128` is a portability trap.** Once you can choose CRT, do; the cost is one parallel ring at compile time and the gain is every other piece of silicon.
- **fp16 K cache must align with HVX 1024-bit vectors** when running on V69; otherwise the kernel falls off the fast path silently. The packed 63-byte block satisfies this exactly.
- **Single-thread first, scale second.** Phase 2.3b's 8-thread run produced bit-identical output to single-thread *after* `memset(scratch, 0)` was added in the per-thread inner loop. Dirty scratch buffers were the root cause of an early 5-nat PPL swing.
- **The norm-weight $+1.0$ offset on Gemma3** is mandatory; missing it explodes the residual stream by 92 orders of magnitude (one of our actual numbers during Phase 2.3 iter 1).

15. Conclusion

The mathematics of Part I executes. On x86, the engine runs at Gemma3-1B scale with six significant figures of bit-exactness between shimmed and unshimmed inference, validating Theorem 4 at production scale. The CRT-NTT kernel removes the last barrier to running the math on devices without 128-bit ALUs and is now bit-identical between Linux GCC and Windows MSVC. The Hexagon V69 backend reaches first-light: $9.78\times$ KV compression, FastRPC engaged, prefill on Qwen3-4B running at 1.67

t/s — the per-K dispatch wall is the next known fix.

Two days of work. The framework is built. The engine runs.

References (system)

1. Shannon-Prime, *project_paths_and_stuff.md* (build environment master doc).
2. Cooley & Tukey (1965), *An algorithm for the machine calculation of complex Fourier series*.
3. Cheon et al. (2017), *Homomorphic encryption for arithmetic of approximate numbers* (CKKS).
4. Barrett, P. (1986), *Implementing the Rivest, Shamir, and Adleman public key encryption algorithm on a standard digital signal processor*.
5. Qualcomm Hexagon SDK V69 documentation.
6. *FastRPC dispatch ground truth* — Shannon-Prime internal memo (577 calls/s ceiling).
7. *KV Cache Is A View v2* — Shannon-Prime internal document.

---

*The companion theoretical paper is Part I. Source code (engine, math core, Hexagon backend) is at `D:\F\shannon-prime-repos` and its three sibling repos. Test results in this paper are reproducible with the CLI in §11.3.